CODE BREAKDOWN- SYSTEM MESSAGE SERVICES XML

queue#SystemMessage

This service is a Moqui service that **queues an outgoing system message** and optionally sends it immediately. Think of it as a message pipeline:

Create message → store in database → optionally trigger sending → update status later

Let's break it line by line and then see a complete real-world flow.

Service definition

<service verb="queue" noun="SystemMessage">

This creates a service named:

queue#SystemMessage

In Moqui:

- `verb` = action (`create`, `update`, `queue`, `send`, etc.)
- `noun` = entity/object (`SystemMessage`)

So this service means:

> "Queue a system message for sending"

Description section

<description>
Queue an outgoing message...
</description>

This explains the behavior:

1. Create a `SystemMessage` record
2. Initial status = `Produced`
3. If `sendNow=true`
    - immediately try sending
4. Otherwise:
    - keep it in queue
    - another scheduler service sends later

Think of it like email:

Write email
↓
Save in Outbox
↓
Send now OR send later

Input parameters

---

systemMessageId

<parameter name="systemMessageId">

Normally Moqui generates sequence IDs automatically.

Example:

SM10001
SM10002
SM10003

But sometimes you need the ID beforehand.

Suppose you're sending:

```
{
  "messageId":"SM10001",
  "orderId":"ORD100"
}
```

External systems may use this as a reference.

So you can pass:

systemMessageId="SM10001"

instead of generating it automatically.

---

auto-parameters

<auto-parameters entity-name="moqui.service.message.SystemMessage" include="nonpk"/>

This automatically imports all non-primary-key fields from:

SystemMessage entity

Instead of writing:

<parameter name="messageText"/>
<parameter name="remoteId"/>
<parameter name="statusId"/>
...

Moqui creates them automatically.

---

systemMessageTypeId

<parameter name="systemMessageTypeId" required="true"/>

Defines message type.

Example:

EMAIL
JSON_RPC
ORDER_SYNC
SMS

Suppose:

systemMessageTypeId="ORDER_SYNC"

This tells Moqui:

"Use order synchronization logic"

---

messageText

<parameter name="messageText" required="true"/>

Actual message content.

Example:

```
{
  "orderId":"ORD100",
  "customer":"John",
  "amount":250
}
```

---

systemMessageRemoteId

<parameter name="systemMessageRemoteId">

Target system identifier.

Example:

systemMessageRemoteId="ERP_SYSTEM"

or

systemMessageRemoteId="WAREHOUSE_API"

Meaning:

Send this message to warehouse system

---

statusId

<parameter name="statusId" default-value="SmsgProduced"/>

Default:

SmsgProduced

Meaning:

Message created
Not sent yet

Possible lifecycle:

SmsgProduced
  ↓
SmsgSending
  ↓
SmsgSent

or

SmsgProduced
  ↓
SmsgError

---

isOutgoing

<parameter name="isOutgoing" default-value="Y"/>

Indicates direction.

Y → outgoing message
N → incoming message

Example:

Outgoing:

Order sent to ERP

Incoming:

ERP sends inventory update

---

initDate

<parameter
  name="initDate"
  type="Timestamp"
  default="ec.user.nowTimestamp"/>

Stores creation timestamp.

Example:

2026-05-22 18:35:20

---

sendNow

<parameter name="sendNow"
type="Boolean"
default="true"/>

Controls whether message should send immediately.

Case 1:

sendNow=true

Flow:

Create message
↓
Send immediately

Case 2:

sendNow=false

Flow:

Create message
↓
Store in queue
↓
Scheduler sends later

---

mode

<parameter name="mode"
default-value="async"/>

Defines sending style.

Possible:

async
sync

Output

```
<out-parameters>
  <parameter name="systemMessageId"/>
</out-parameters>
```

Returns generated message ID.

Example:

systemMessageId=SM10045

Actions section

Now the real work begins.

Step 1: Create database record

```
<service-call
    name="create#moqui.service.message.SystemMessage"
    in-map="context"
    out-map="context"
    transaction="force-new"/>
```

This inserts into database.

Equivalent SQL:

```
INSERT INTO SystemMessage
(
systemMessageId,
messageText,
statusId,
systemMessageTypeId
)
VALUES
```

(
'SM10045',
'Order JSON',
'SmsgProduced',
'ORDER_SYNC'
);

`transaction="force-new"`

means:

Start separate transaction
Commit immediately

Even if parent service fails later:

message still exists

Step 2: Check sendNow

<if condition="sendNow">

If:

sendNow=false

Stop here.

Message remains:

SmsgProduced

Scheduler handles later.

Step 3: Check mode

<if condition="mode == async">

Two paths:

Async mode

```
<service-call
  name="org.moqui.impl.SystemMessageServices.send#ProducedSystemMessage"
  in-map="[systemMessageId:systemMessageId]"
  async="true"/>
```

Flow:

Create message
↓
Return immediately
↓
Background thread sends message

User doesn't wait.

Example:

Customer places order:

Save order
↓
Queue message
↓
Return success immediately
↓
Background process sends ERP update

Very common in production.

Sync mode

```
<service-call
name="org.moqui.impl.SystemMessageServices.send#ProducedSystemMessage"
ignore-error="true"
transaction="force-new"/>
```

Flow:

Create message
↓

Wait
↓
Send immediately
↓
Return result

---

ignore-error="true"

Means:

Do not crash parent service if sending fails

Example:

Without ignore-error:

Order created
↓
Message send failed
↓
Whole transaction rollback

Bad.

With ignore-error:

Order created
↓
Message send failed
↓
Order remains
↓
Message status updated to error

Complete real-world example

Suppose an order is placed:

```
<service-call
  name="queue#SystemMessage"
  in-map="[
     systemMessageTypeId:'ORDER_SYNC',
```

```
    messageText:'{
       orderId:ORD100,
       amount:500
    }',
    systemMessageRemoteId:'ERP',
    sendNow:true,
    mode:'async'
]"/>
```

Execution flow:

Step 1

Create DB row:

ID	Type	Status
SM10045	ORDER_SYNC	SmsgProduced

Step 2

Since:

sendNow=true

continue.

Step 3

Since:

mode=async

start background send.

Step 4

Background service executes:

send#ProducedSystemMessage

Step 5

Calls actual integration service:

ERP API

Step 6

Status changes:

SmsgProduced
↓
SmsgSending
↓
SmsgSent

or

SmsgProduced
↓
SmsgError

Overall architecture:

queue#SystemMessage
↓
Create SystemMessage record
↓
Status = Produced
↓
sendNow?
/ \
No Yes
↓ ↓
Scheduler mode?
/ \
async sync
↓ ↓
background immediate
send send

↓
update status

This service is essentially Moqui's **Outbox Pattern implementation**—store messages reliably in the database first, then send them. That prevents losing messages if a server crashes during processing.

---

send#AllProducedSystemMessages Service – Detailed Explanation

<service verb="send" noun="AllProducedSystemMessages" authenticate="anonymous-all">

Service name:

send#AllProducedSystemMessages

Purpose:

This service is designed to run on a schedule and process all outgoing messages that are currently waiting in `SmsgProduced` status.

Main responsibility:

Find all pending outgoing messages
↓
Check whether enough retry time has passed
↓
Attempt sending again
↓
If retry limit exceeded

↓
Move message to Error status

Think of it as a delivery truck that periodically checks the warehouse and delivers packages.

Warehouse
↓
Package waiting
↓
Delivery truck arrives
↓
Attempt delivery
↓
Success → Delivered
Failure → Retry later
Too many failures → Error

Service Description

<description>
Meant to be run scheduled...
</description>

This service:

1. Runs periodically (scheduler)
2. Finds outgoing messages (`isOutgoing=Y`)
3. Finds messages in `SmsgProduced` state
4. Retries failed messages
5. Stops retrying after limit is reached
6. Marks permanently failed messages as `SmsgError`

Authentication

authenticate="anonymous-all"

Purpose:

Allows scheduler/background jobs to execute the service without requiring a logged-in user.

Reason:

Scheduled services run automatically.

Example:

Every 10 minutes:
Scheduler calls service

No user login needed.

Input Parameters

1. retryMinutes

```
<parameter name="retryMinutes"
        type="BigDecimal"
        default="60"/>
```

Purpose:

Defines waiting time before retrying failed messages.

Default:

60 minutes

Meaning:

If previous attempt happened less than 60 minutes ago
↓
Do not retry

Example:

Current time:

5:00 PM

Last attempt:

4:30 PM

Difference:

30 minutes

Since:

30 < 60

Message skipped.

---

2. retryLimit

```
<parameter name="retryLimit"
        type="Integer"
        default="24"/>
```

Purpose:

Maximum retry attempts.

Default:

24 attempts

Comment:

<!-- by default try for 1 day -->

Meaning:

Retry every hour
24 retries
= 24 hours

Example:

Attempt 1 → Failed
Attempt 2 → Failed
Attempt 3 → Failed
...
Attempt 24 → Failed
Attempt 25
↓
Move to Error status

---

3. systemMessageTypeIds

<parameter name="systemMessageTypeIds"
type="List"/>

Purpose:

Filter specific message types.

Example:

systemMessageTypeIds=[
"ORDER_SYNC",
"INVENTORY_SYNC"
]

Meaning:

Process only these message types

Without this:

Process all types

---

4. mode

```
<parameter name="mode"
        default-value="async"/>
```

Possible values:

async
sync

Purpose:

Determines how sending happens.

Actions Section

Actual processing begins here.

Step 1: Calculate Retry Timestamp

Code:

```
<set field="retryTimestamp"
from="new Timestamp(
(System.currentTimeMillis()
-(retryMinutes * 60000))
as long)"/>
```

Purpose:

Calculates time before which retry is allowed.

Formula:

Current Time − Retry Minutes

Example:

Current time:

6:00 PM

Retry minutes:

60

Calculation:

6:00 PM − 60 minutes

Result:

retryTimestamp = 5:00 PM

Meaning:

Only retry messages whose last attempt
was before 5:00 PM

Step 2: Find Eligible Messages

Code:

```
<entity-find entity-name="moqui.service.message.SystemMessage"
        list="smList"
        limit="200">
```

Purpose:

Retrieve messages eligible for sending.

Maximum:

200 messages at once

Reason:

Avoid system overload.

Condition 1

```
<econdition field-name="statusId"
        value="SmsgProduced"/>
```

Meaning:

Only messages waiting to be sent

Condition 2

```
<econdition field-name="isOutgoing"
        value="Y"/>
```

Meaning:

Only outgoing messages

Ignore:

Incoming messages

Condition 3

```
<econdition field-name="lastAttemptDate"
        operator="less"
        from="retryTimestamp"
        or-null="true"/>
```

Meaning:

lastAttemptDate < retryTimestamp
OR
lastAttemptDate is null

Example:

Current:

6:00 PM

Retry timestamp:

5:00 PM

Messages:

Message A → lastAttempt=4:00 PM
Message B → lastAttempt=5:45 PM
Message C → null

Result:

A → selected
B → skipped
C → selected

Condition 4

<econdition field-name="systemMessageTypeId"
operator="in"
from="systemMessageTypeIds"
ignore-if-empty="true"/>

Meaning:

If list exists:

Only selected message types

If empty:

Ignore condition

Order By

<order-by field-name="initDate"/>

Purpose:

Oldest messages processed first.

Example:

SM1001 → 10:00
SM1002 → 10:05
SM1003 → 10:10

Processing:

SM1001
SM1002
SM1003

FIFO behavior.

Step 3: Iterate Through Messages

Code:

<iterate list="smList"
entry="sm">

Purpose:

Process each message one by one.

Equivalent Java:

for(SystemMessage sm : smList)

Step 4: Check Retry Limit

Code:

<if condition="sm.failCount < retryLimit">

Purpose:

Check whether retries are still allowed.

Example:

retryLimit=24
failCount=10

Result:

Retry allowed

Example:

retryLimit=24
failCount=25

Result:

No retry
Move to error

Async Mode

Code:

```
<service-call
name="org.moqui.impl.SystemMessageServices.send#ProducedSystemMessage"
in-map="[systemMessageId:sm.systemMessageId]"
async="true"/>
```

Flow:

```
Pick message
    ↓
Start background send
    ↓
Continue immediately
```

Advantages:

Fast
Handles many messages
Does not block processing

Sync Mode

Code:

```
<service-call
name="org.moqui.impl.SystemMessageServices.send#ProducedSystemMessage"
ignore-error="true"
transaction="force-new"/>
```

Flow:

```
Pick message
    ↓
Wait for sending
    ↓
Continue
```

Retry Limit Exceeded

Code:

```
<service-call
name="update#moqui.service.message.SystemMessage"
transaction="force-new"
in-map="[
systemMessageId:sm.systemMessageId,
statusId:'SmsgError',
lastAttemptDate:ec.user.nowTimestamp
]"/>
```

Purpose:

Move permanently failing message to error state.

Update:

statusId=SmsgError

Example:

Before:

```
SM1001
failCount=24
status=SmsgProduced
```

After:

```
SM1001
failCount=24
status=SmsgError
```

Meaning:

Stop retrying permanently

Complete Real World Example

Suppose database contains:

Message ID	Status	Fail Count	Last Attempt
SM1001	SmsgProduced	2	3:00 PM
SM1002	SmsgProduced	10	5:30 PM
SM1003	SmsgProduced	25	2:00 PM

Current time:

6:00 PM

RetryMinutes:

60

RetryTimestamp:

5:00 PM

Processing:

SM1001

3:00 PM < 5:00 PM
failCount=2

Result:

Send message

SM1002

5:30 PM > 5:00 PM

Result:

Skip

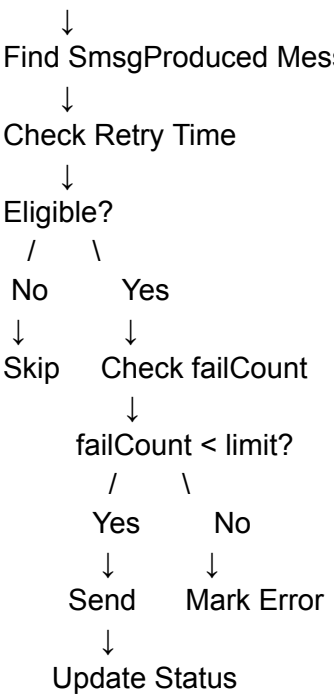
SM1003

failCount=25
retryLimit=24

Result:

Move to SmsgError

Complete Flow Diagram



Overall Purpose

This service acts like a retry engine for message processing.

Responsibilities:

1. Finds pending messages
2. Retries failed messages
3. Prevents rapid repeated retries
4. Stops infinite retries
5. Marks permanently failing messages as errors
6. Supports background processing

Relationship with previous service:

queue#SystemMessage
↓
Create message
↓
Status=SmsgProduced
↓
send#AllProducedSystemMessages
↓
Retry and process messages
↓
SmsgSent or SmsgError

This is essentially the scheduled retry mechanism of the Outbox Pattern in Moqui.

---

receive#SystemMessage Interface – Detailed Explanation

<service verb="receive" noun="SystemMessage" type="interface">

Service name:

receive#SystemMessage

Type:

Interface Service

Purpose:

This is not an actual implementation service that executes logic.

Instead, it defines a contract (input/output structure) that other services implementing it must follow.

Think of it like a template:

Interface
↓
Defines what data is expected
↓
Actual services implement it

Real-world analogy:

Job Application Form
↓
Required fields:
Name
Email
Phone
Address
↓
Different companies use the same form

The form itself does not do anything.

It only defines what information is needed.

Similarly:

receive#SystemMessage

defines:

- what data incoming messages must provide
- what output should be returned

Why is this interface overridden?

Comment:

<!-- Overriding the receive#SystemMessage interface
to include productStoreId -->

Reason:

Original interface did not contain:

productStoreId

But:

receive#IncomingSystemMessage

creates a `SystemMessage` record.

The requirement was:

Incoming messages should also store productStoreId

Therefore:

Interface overridden

to add:

<parameter name="productStoreId"/>

Flow Before Override

Original flow:

Incoming Message
↓
receive#IncomingSystemMessage
↓
Create SystemMessage

Stored:

systemMessageId
messageText
systemMessageTypeId
remoteMessageId

Missing:

productStoreId

Problem:

Cannot identify which product store
the message belongs to

Flow After Override

Incoming Message

↓
receive#IncomingSystemMessage
↓
receive#SystemMessage Interface
↓
Create SystemMessage

Stored:

systemMessageId
messageText
productStoreId
remoteMessageId

Now:

Message linked to correct store

Input Parameters

---

1. systemMessageTypeId

```
<parameter name="systemMessageTypeId"
        required="true"/>
```

Purpose:

Defines message type.

Examples:

ORDER
INVENTORY
FULFILLMENT
SHOPIFY_WEBHOOK

Example:

systemMessageTypeId="SHOPIFY_ORDER"

Meaning:

Incoming Shopify order message

---

2. messageText

<parameter name="messageText"
required="true"/>

Purpose:

Contains actual message payload.

Example:

```
{
  "orderId":"1001",
  "customer":"John"
}
```

---

3. systemMessageRemoteId

<parameter name="systemMessageRemoteId"/>

Purpose:

Identifies external source system.

Examples:

SHOPIFY
ERP
WMS
AMAZON

Example:

systemMessageRemoteId="SHOPIFY"

Meaning:

Message came from Shopify

---

4. remoteMessageId

<parameter name="remoteMessageId"/>

Purpose:

Stores external system's message identifier.

Example:

remoteMessageId="SH100234"

Useful for:

Tracking
Duplicate detection
Debugging

---

5. productStoreId

<parameter name="productStoreId"/>

Purpose:

Stores related product store.

Example:

productStoreId="STORE100"

Meaning:

Message belongs to STORE100

Why important:

Suppose company has:

STORE_A
STORE_B
STORE_C

Incoming message:

```
{
  "orderId":"1001"
}
```

Without:

productStoreId

System cannot determine:

Which store owns order?

With:

productStoreId="STORE_A"

Now system knows.

---

6. consumeSmrId

<parameter name="consumeSmrId"/>

Purpose:

Added later for new fulfillment functionality.

The comments explain a deployment issue.

Problem Explained

Two components were overriding same interface:

Component 1:

mantle-shopify-connector

Added:

<parameter name="consumeSmrId"/>

Component 2:

ofbiz-oms-usl

Added:

<parameter name="productStoreId"/>

Expected:

Merged result:

<parameter name="consumeSmrId"/>
<parameter name="productStoreId"/>

Actual result:

Only:

<parameter name="productStoreId"/>

was added.

Why did this happen?

Based on comments:

Component deployment order
(alphabetical order)

Suppose:

mantle-shopify-connector
deployed first

Then:

ofbiz-oms-usl
deployed later

Second deployment overrides first.

Result:

Earlier changes lost

Expected behavior:

Merge fields

Actual behavior:

Replace fields

So temporary solution:

Add consumeSmrId manually

---

7. parentMessageId

<parameter name="parentMessageId"/>

Purpose:

Stores parent-child relationship between messages.

Example:

Parent message:

SM100

Generated child messages:

SM101
SM102
SM103

Relationship:

SM100
  ↓

↓ ↓ ↓
SM101 SM102 SM103

Useful for:

Message tracking
Grouping
Debugging

Output Parameters

```
<out-parameters>
   <parameter name="systemMessageIdList"
            type="List">
        <parameter name="systemMessageId"/>
   </parameter>
</out-parameters>
```

Output:

Returns list of created message IDs.

Example:

```
[
 SM1001,
 SM1002,
 SM1003
]
```

Why list instead of single value?

Because one incoming message may generate multiple system messages.

Example:

Incoming order:

ORDER100

Generates:

SM1001 → Inventory Message
SM1002 → Payment Message
SM1003 → Shipping Message

Output:

```
systemMessageIdList:
[
SM1001,
SM1002,
SM1003
]
```

Complete Real World Flow

Suppose Shopify sends:

```
{
  "orderId":"ORD100",
  "store":"STORE_A",
  "messageId":"SH1001"
}
```

Flow:

```
Shopify Webhook
      ↓
receive#IncomingSystemMessage
      ↓
receive#SystemMessage Interface
      ↓
Create SystemMessage Record
```

Stored values:

```
systemMessageId = SM1001
systemMessageTypeId = SHOPIFY_ORDER
messageText = Order JSON
systemMessageRemoteId = SHOPIFY
remoteMessageId = SH1001
productStoreId = STORE_A
```

Output:

```
systemMessageIdList:
[
SM1001
]
```

Overall Architecture

```
External System
   ↓
Incoming Message
```

↓
receive#SystemMessage (Interface)
↓
Defines required fields
↓
receive#IncomingSystemMessage
↓
Create SystemMessage
↓
Return message IDs

---

consume#AllReceivedSystemMessages Service – Detailed Explanation

<service verb="consume" noun="AllReceivedSystemMessages" authenticate="anonymous-all">

Service name:

consume#AllReceivedSystemMessages

Purpose:

This service is a scheduled retry service for incoming messages.

It finds all incoming `SystemMessage` records that:

- are received but not yet consumed
- previously failed during processing
- are ready for retry

Then it retries processing them.

Flow:

Incoming Message Received
↓
Status = SmsgReceived
↓
Consumption failed
↓
Stored for retry
↓
consume#AllReceivedSystemMessages
↓
Retry consuming
↓
Success → SmsgConsumed
Failure → Retry again
Too many failures → SmsgError

Think of it as a retry worker for incoming messages.

Example:

Warehouse receives package
↓
Worker tries processing
↓
Processing failed
↓
Keep package aside
↓
Retry worker comes later
↓
Try processing again

Description

<description>
Consume incoming (isOutgoing=N)
SystemMessage records not already consumed
</description>

Purpose:

1. Find incoming messages
2. Find messages in `SmsgReceived`
3. Retry failed consumption
4. Retry until retry limit
5. Move to error if retries exhausted

Authentication

authenticate="anonymous-all"

Purpose:

Allows scheduler/background jobs to run without login.

Example:

Every 10 minutes
↓
Scheduler triggers service

No user needed.

Input Parameters

---

1. retryMinutes

```
<parameter name="retryMinutes"
        type="BigDecimal"
        default="10"/>
```

Purpose:

Waiting time before retrying.

Default:

10 minutes

Meaning:

If previous attempt happened less than
10 minutes ago
↓
Skip retry

Example:

Current time:

6:00 PM

Last attempt:

5:55 PM

Difference:

5 minutes

Result:

Skip

---

2. retryLimit

```
<parameter name="retryLimit"
        type="Integer"
        default="3"/>
```

Purpose:

Maximum retry attempts.

Default:

3 attempts

Example:

Attempt 1 → Fail
Attempt 2 → Fail
Attempt 3 → Fail
Attempt 4
↓
Move to Error

---

3. systemMessageTypeIds

<parameter name="systemMessageTypeIds"
type="List"/>

Purpose:

Process only selected message types.

Example:

systemMessageTypeIds=[
"SHOPIFY_ORDER",
"INVENTORY_UPDATE"
]

Meaning:

Retry only these message types

If empty:

Retry all types

---

4. mode

```
<parameter name="mode"
        default-value="async"/>
```

Possible values:

async
sync

Purpose:

Controls retry processing style.

---

5. limit

```
<parameter name="limit"
        type="Integer"
        default-value="200"/>
```

Purpose:

Maximum records processed at once.

Default:

200 messages

Reason:

Prevents overload.

Actions Section

Actual processing starts here.

Step 1: Calculate Retry Timestamp

Code:

```
<set field="retryTimestamp"
from="new Timestamp(
(System.currentTimeMillis()
-(retryMinutes * 60000))
as long)"/>
```

Formula:

Current Time − Retry Minutes

Example:

Current:

6:00 PM

RetryMinutes:

10

Calculation:

6:00 PM − 10 min

Result:

retryTimestamp=5:50 PM

Meaning:

Retry only messages attempted
before 5:50 PM

Step 2: Find Eligible Messages

Code:

<entity-find
entity-name="moqui.service.message.SystemMessage"
list="smList"
limit="limit">

Purpose:

Fetch messages eligible for retry.

Condition 1

<econdition field-name="statusId"
value="SmsgReceived"/>

Meaning:

Only received but not consumed messages

Example:

SmsgReceived → Selected
SmsgConsumed → Ignored
SmsgSent → Ignored

Condition 2

<econdition field-name="isOutgoing"
value="N"/>

Meaning:

Only incoming messages

Values:

Y → Outgoing
N → Incoming

Condition 3

<econdition field-name="lastAttemptDate"
operator="less"
from="retryTimestamp"
or-null="true"/>

Meaning:

lastAttemptDate < retryTimestamp
OR
lastAttemptDate is null

Example:

Current:

6:00 PM

Retry timestamp:

5:50 PM

Messages:

SM1001 → 5:20 PM
SM1002 → 5:57 PM
SM1003 → null

Result:

SM1001 → Selected
SM1002 → Skipped
SM1003 → Selected

Condition 4

<econdition field-name="systemMessageTypeId"
operator="in"
from="systemMessageTypeIds"
ignore-if-empty="true"/>

Meaning:

If list exists:

Filter by message type

Otherwise:

Ignore filter

Order By

<order-by field-name="initDate"/>

Purpose:

Oldest messages processed first.

FIFO behavior.

Example:

SM1001 → 10:00
SM1002 → 10:05
SM1003 → 10:10

Processing:

SM1001
SM1002

SM1003

Step 3: Iterate Through Messages

Code:

```
<iterate list="smList"
        entry="sm">
```

Equivalent Java:

```
for(SystemMessage sm : smList)
```

Purpose:

Process messages one by one.

Step 4: Check Retry Limit

Code:

```
<if condition="sm.failCount < retryLimit">
```

Purpose:

Check whether retries remain.

Example:

```
retryLimit=3
failCount=2
```

Result:

Retry allowed

Example:

retryLimit=3
failCount=5

Result:

Move to Error

Async Mode

Code:

```
<service-call
name="org.moqui.impl.SystemMessageServices.consume#ReceivedSystemMessage"
in-map="[systemMessageId:sm.systemMessageId]"
async="true"/>
```

Flow:

Pick message
↓
Start background consumption
↓
Continue immediately

Advantages:

Fast processing
Better throughput
No waiting

Sync Mode

Code:

```
<service-call
name="org.moqui.impl.SystemMessageServices.consume#ReceivedSystemMessage"
transaction="force-new"
ignore-error="true"/>
```

Flow:

```
Pick message
    ↓
Wait for consumption
    ↓
Continue
```

Explanation:

transaction="force-new"

Start separate transaction

ignore-error="true"

Do not stop entire process if one message fails

Retry Limit Exceeded

Code:

```
<service-call
name="update#moqui.service.message.SystemMessage"
transaction="force-new"
in-map="[
systemMessageId:sm.systemMessageId,
statusId:'SmsgError',
lastAttemptDate:ec.user.nowTimestamp
]"/>
```

Purpose:

Move permanently failing messages to error state.

Before:

SM1001
failCount=4
status=SmsgReceived

After:

SM1001
failCount=4
status=SmsgError

Meaning:

Stop retrying permanently

Complete Real World Example

Database:

Message ID	Status	Fail Count	Last Attempt
SM1001	SmsgReceived	1	5:20 PM
SM1002	SmsgReceived	2	5:57 PM
SM1003	SmsgReceived	4	5:00 PM

Current time:

6:00 PM

RetryMinutes:

10

RetryTimestamp:

5:50 PM

Processing:

SM1001

5:20 PM < 5:50 PM
failCount=1

Result:

Retry consumption

SM1002

5:57 PM > 5:50 PM

Result:

Skip

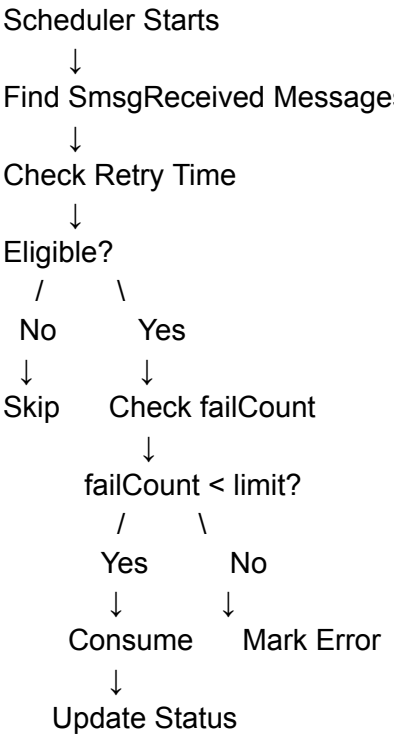
SM1003

failCount=4
retryLimit=3

Result:

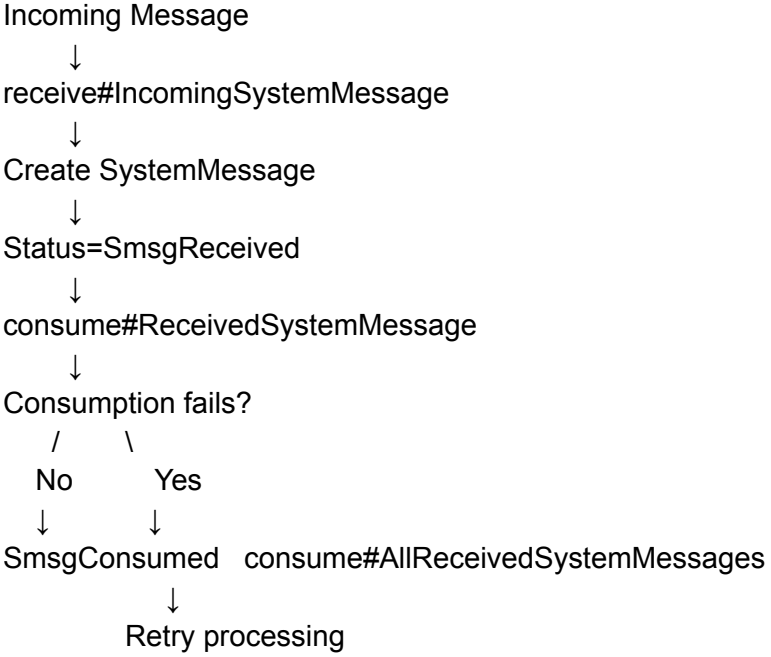
Move to SmsgError

Complete Flow Diagram



Relationship With Previous Services

Full message lifecycle:



↓
SmsgConsumed/SmsgError

Important Learning Points

1. Handles retry of incoming messages
2. Processes only:

status=SmsgReceived
isOutgoing=N

3. Prevents immediate repeated retries
4. Supports async and sync modes
5. Stops infinite retries
6. Moves permanent failures to `SmsgError`
7. Works as retry engine for incoming message processing
8. Complements `send#AllProducedSystemMessages`

Difference:

send#AllProducedSystemMessages
↓
Retries outgoing messages

consume#AllReceivedSystemMessages
↓
Retries incoming messages

This is the incoming-side retry mechanism of the System Message framework.